Basic (your first real API call) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The three pieces, and what each returns
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The three pieces, and what each returns

Three pieces, matching the Core Concepts above: `client = OpenAI()` creates the client (it reads `OPENAI_API_KEY` from the environment on its own, so call `load_dotenv()` from Doc01's config pattern first if you haven't already). `client.chat.completions.create(model=..., messages=[...])` sends the request. The `messages` list needs at least one `{"role": "system", ...}` entry and one `{"role": "user", ...}` entry — the system entry sets behavior, the user entry is the actual question.

`response.choices` is a list because the API can technically return more than one candidate answer (controlled by an `n` parameter you're not using here) — with the default settings, there's exactly one, at index `0`. `response.choices[0].message.content` is a plain string: the model's reply text.

For the system-prompt comparison, keep the user message identical across both runs and change only the system message — that isolates the variable you're actually testing.

Hint 2 — The plan, and almost the whole thing

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

function ask(system_prompt, user_prompt) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
```

```
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content

print(ask("You are formal and professional.", "Tell me about your day."))
print(ask("You are a sarcastic pirate.", "Tell me about your day."))
```

Wrapping the call in a small `ask()` function is optional for this exercise but makes comparing two system prompts much less repetitive. Write it this way, run it, confirm the two replies clearly differ in tone, then compare against the Solution.

Then move each system prompt into a file (`prompts/formal.txt`) and write `load_prompt(name)` : `with open(path) as prompt_file:` then `return prompt_file.read().strip()` . Use `name.lower()` in the path, so `"Formal"` still finds `formal.txt` .

Full solution: Show me the solution